

Cybersecurity for embedded systems

REPORT

F214426

Part 1: Deployment and performance assessment of cryptography algorithms

This section analyzes and compares the performance of three crypto-algorithms, AES, Blowfish and XTEA on the development board Nucleo-F767ZI. The comparison is made based on two of the most significant parameters that are highly critical in embedded system scenarios: inference (runtime) time and memory consumption. These algorithms were tested at two levels of compiler optimization: -O0 (optimization off) and -O3 (optimization on), in order to understand how optimizations made at the compiler level influence their size and performance on restricted hardware.

To effectively measure execution time, a dedicated microsecond-resolution timer function named `getCurrentMicros()` was introduced. The function exploits the `SysTick` timer and system tick count to provide high-resolution timing, which is essential in benchmarking short cryptographic procedures. The execution time measured is also for only encryption. Although by running the code provided, the decryption time can also be measured (it is seen to be the same).

```
FW HASH Result:
13ba72d32a073077bb625bad638f29def5c444f09cda82557e97d2e05c509b7f
Expected HASH Result:
13ba72d32a073077bb625bad638f29def5c444f09cda82557e97d2e05c509b7f

FW Hash check pass
Encrypted: DF333FD2 30A71BB4
Encryption time: 58429 us
Decrypted: 00000001 00000002
Decryption time: 10 us

Start FW Hash Check...
FW start address: 0x08000000
FW size: 0x00007000
FW HASH address: 0x08007000
FW HASH SIZE: 0x00000020

FW HASH Result:
13ba72d32a073077bb625bad638f29def5c444f09cda82557e97d2e05c509b7f
Expected HASH Result:
13ba72d32a073077bb625bad638f29def5c444f09cda82557e97d2e05c509b7f

FW Hash check pass
Encrypted: DF333FD2 30A71BB4
Encryption time: 58429 us
Decrypted: 00000001 00000002
Decryption time: 10 us
```

Figure 1- example for encryption and decryption time calculation

As per the data collected and their respective performance graphs, it was clearly evident that the XTEA algorithm consistently depicted smaller execution

times and memory usage, especially in -O0 optimization. The lightweight cipher was most effective in RAM and Flash footprint and therefore ideally suited for microcontroller-based systems where processing power and memory are limited.

However, the optimization level was increased to -O3, and Blowfish's run time showed a remarkable increase while its raw inference time remained better compared to XTEA. Nevertheless, Blowfish's Flash memory footprint remained much higher due to its use of large substitution boxes and more intricate key scheduling algorithms. AES, although the gold standard for modern-day encryption, saw the highest memory utilization and slower performance in both the optimization cases—showcasing its computational intensity for its 128-bit block and key expansion step.

Model	Compilation flag	Av. execution time	Memory utilisation	
			Flash	RAM
XTEA	O0	39us	19.41Kb	3.31Kb
	O3	14us	15.41Kb	3.31Kb
Blowfish	O0	44.5us	23.72Kb	3.31Kb
	O3	10us	23.68Kb	3.31Kb
XTEA + firmware integrity (SHA256)	O0	60558us	22.05Kb	3.45Kb
	O3	53405us	18.18Kb	3.45Kb
Blowfish + firmware integrity (SHA256)	O0	57760us	26.23Kb	3.31Kb
	O3	58429us	26.03Kb	3.31Kb

Figure 2- Part one value table

Model	Compilation flag	Av. execution time	Memory utilisation	
			Flash	RAM
AES	O0	461us	31.45Kb	3.67Kb
	O3	320us	25.08Kb	3.67Kb

Figure 3- AES values

Based on these observations, XTEA eventually was selected for inclusion in the final implementation in Part 2. Its low usage of resources, combined with its sufficient cryptographic strength for all but the most demanding embedded uses, made it

the best pragmatic solution for ensuring efficient and secure execution on the STM32F7 series MCU. In real application in systems such as drone systems, where energy usage and real-time calculation are critical, this security strength vs. computational cost trade-off is a critical design consideration.

The performance and low memory requirements of XTEA algorithm aligns with the embedded system constraints and demands. In embedded or power limited systems like these drones, XTEA would be more ideal if speed and low memory are issues. While AES is more secure, it could increase the latency as well as the memory overhead.

Part 2: Mission validation

The drone operation is separated into four main mission phases and is designed to ensure confidentiality and integrity throughout the communication process between ground station and the drone.

Boot and Firmware Verification Phase:

The mission starts with the boot and authentication stage. The drone system is set to be in an active waiting stage and will be listening for a “start” command from the ground station on the UART interface. Once it receives the command, it implies the initiation of communication between the drone and ground station and the firmware runs a critical safety function “FW_SIGNATURE_VERIFY”. This function performs a digital signature verification of the firmware. The ECC signature that is programmed into the signature section is checked against a prestored public key to ensure that the firmware has not been modified or tampered with. Once this verification passes, the firmware fulfils level 3 security requirements.

```

"filesize is 28232"
"page size is 2048"
"Size after padding is 28672"
##### Generate hash #####
FW HASH
000000 ff f0 a2 59 04 9d d5 4f 10 d4 a2 6d 92 8d 41 f9
000010 7c 4b 47 04 cf 09 b3 e8 d4 f2 46 32 a6 a5 22 b4
##### Sign hash biary #####
##### Convert signature from asn1 to raw binary #####
FW Signature
000000 fa f1 78 05 3c 27 83 7d 71 22 4e 83 b7 c0 a0 62
000010 b4 14 4b bf 12 42 92 4f 66 e1 93 67 15 91 00 74
000020 a5 f0 98 f3 50 22 00 0c 9c d0 de 8a 35 7e 87 41
000030 98 1e 30 69 1a 9a c7 87 b8 9a 86 ee 5b 00 10 45
"##### Generate public key from private ecc key "
"##### Convert public key to binary "
read EC key
Public Key
000000 14 45 ef 01 dd 24 0b de 92 fc 82 30 10 db 01 7d
000010 72 67 fc 59 7e 51 39 b4 c1 ad 79 f3 95 d1 94 8d
000020 ce 63 dc 2d 8e 21 54 a5 a9 90 fd 2c 91 90 80 40
000030 3a db ad 07 b6 bc 86 d5 8e 59 ec da 0b 15 a0 44
##### Generate bin + padding + hash #####
##### Generate bin + padding + hash + sig #####
##### Generate bin + padding + hash + sig + pubkey #####
Binary with Signature to be flashed:
"C:\Users\akash\Downloads\Host_Tools\Host_Tools\out_with_sig_kpub.bin"
Press any key to continue . . .

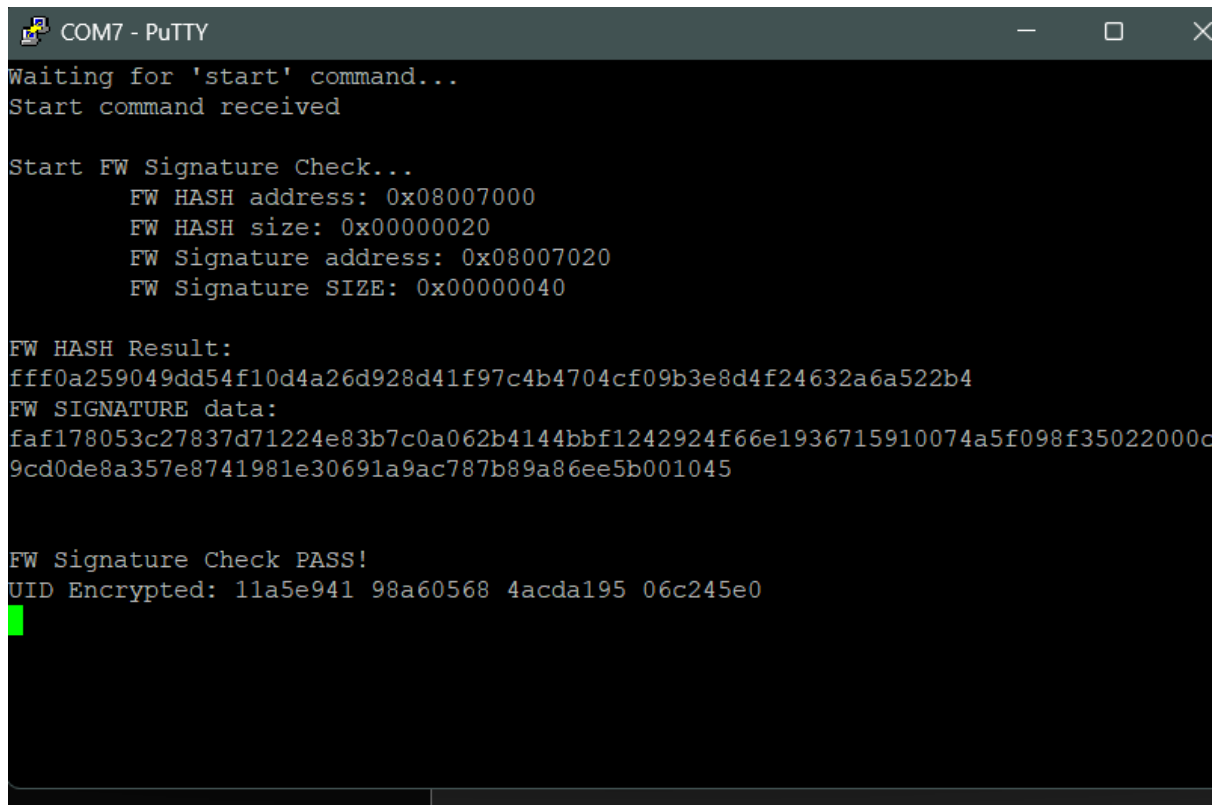
```

Figure 4- creation of signature to be programmer into the board

Encrypted Board Identification:

After the firmware's authenticity is verified, the drone then runs another function that identifies itself to the ground system. This is done by reading the board's unique hardware identifier (

UID) from the microcontroller's memory. This UID is composed of three 32 bit values that are able to uniquely identify each board in production. The firmware then encrypts this value and sends it to the ground station using the XTEA algorithm. Once the ground station receives the data, it then decrypts the message using the same shared key and thus confirms the identity of the drone.



```
COM7 - PuTTY
Waiting for 'start' command...
Start command received

Start FW Signature Check...
    FW HASH address: 0x08007000
    FW HASH size: 0x00000020
    FW Signature address: 0x08007020
    FW Signature SIZE: 0x00000040

FW HASH Result:
fff0a259049dd54f10d4a26d928d41f97c4b4704cf09b3e8d4f24632a6a522b4
FW SIGNATURE data:
faf178053c27837d71224e83b7c0a062b4144bbf1242924f66e1936715910074a5f098f3502200c9cd0de8a357e8741981e30691a9ac787b89a86ee5b001045

FW Signature Check PASS!
UID Encrypted: 11a5e941 98a60568 4acda195 06c245e0
█
```

Figure 5- signature check followed by sending encrypted UID

Command Reception and Car Plate Encryption:

After the identity verification, the drone then enters a “wait for command” state where it waits for further commands from the ground station. The ground station can utilise to commands to communicate with the drone, them being “plate” and “secret”. When the drone receives the “plate” command, the drone generates a pseudo random 6-digit number using the chip’s hardware RNG. This is used to simulate a vehicle registration plate which is then encrypted and sent to ground station. This mimics a real-life situation where the drone picks up and sends surveillance data back to the operator in encrypted format.

Secret Message Transmission:

The final function that the drone does is initiated by using the command “secret” from the ground station. When the drone receives this command, it encrypts and sends a preset hex encoded secret message “416D617A696E67204F7374205C6F2F20” to the ground station. The encrypted message is then decrypted by the ground station.

```
COM7 - PuTTY
Waiting for 'start' command...
Waiting for 'start' command...
Start command received

Start FW Signature Check...
    FW HASH address: 0x08007000
    FW HASH size: 0x00000020
    FW Signature address: 0x08007020
    FW Signature SIZE: 0x00000040

FW HASH Result:
fff0a259049dd54f10d4a26d928d41f97c4b4704cf09b3e8d4f24632a6a522b4
FW SIGNATURE data:
faf178053c27837d71224e83b7c0a062b4144bbf1242924f66e1936715910074a5f098f35022000c
9cd0de8a357e8741981e30691a9ac787b89a86ee5b001045

FW Signature Check PASS!
UID Encrypted: 11a5e941 98a60568 4acda195 06c245e0
Received command: 'secret'
Encrypted secret: 0299b191 8abfb6d5 dc70d874 423d6b24
Received command: 'plate'
Encrypted Plate: c4e16cd4 da7ba8c3
█
```

Figure 6- output from the drone seen from putty

Firmware finalisation and flash programming

There are some components that are flashed onto the board using the STM32 Cube Programmer such as the SHA256 hash digest (for verifying firmware integrity), the digital signature (for verification), and the attendant ECC public key (for verification of the signature). These can be found out using the .bin file that is outputted after the program is run once. This is then put in the Host_Tools folder and a function is called to get these values. These are then put at the end of the firmware. Write protection is then enabled in that section of firmware so that the data cannot be altered and finally read out protection (level BB) is enabled so that no one can see what the data is inside the board.

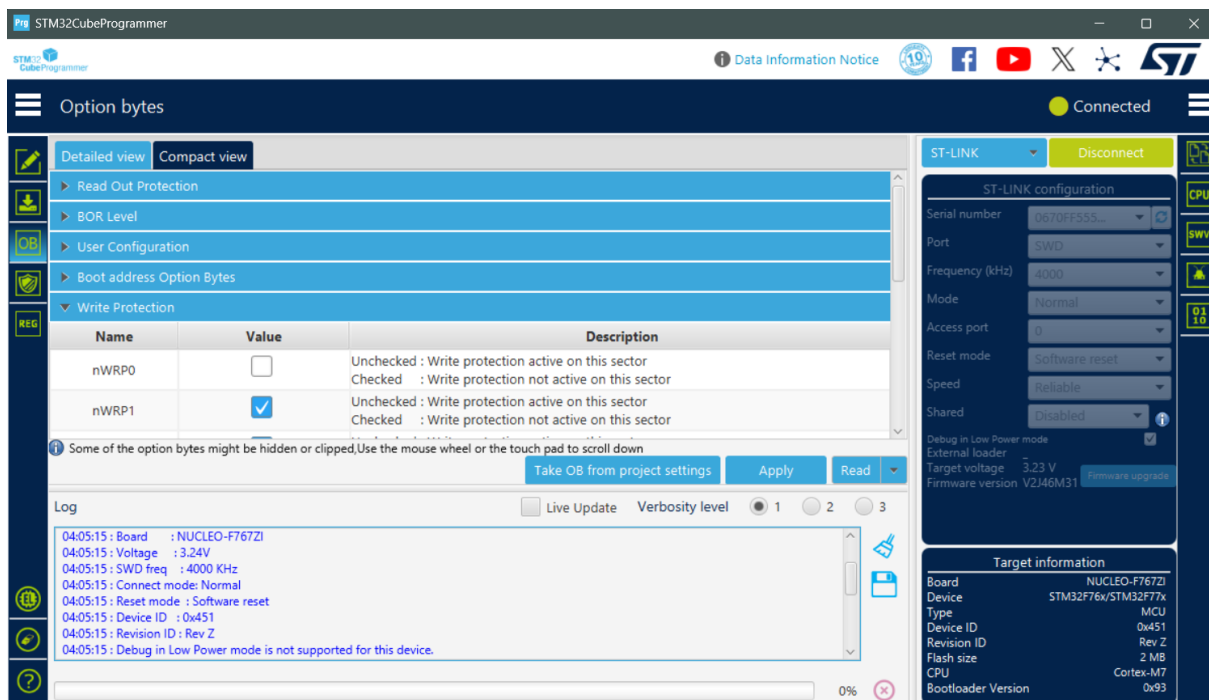


Figure 7- Write protection

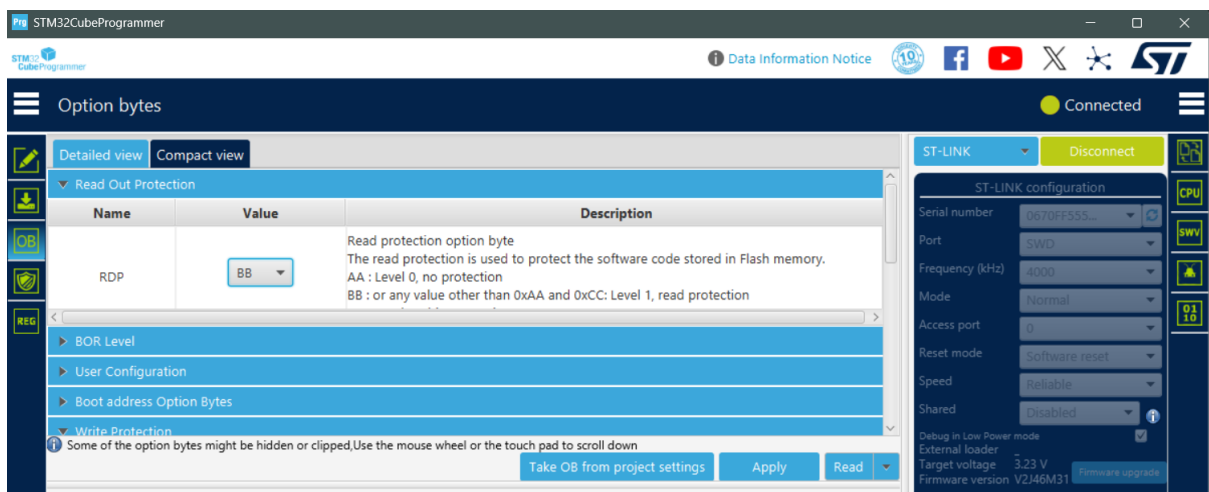


Figure 8- Read out protection

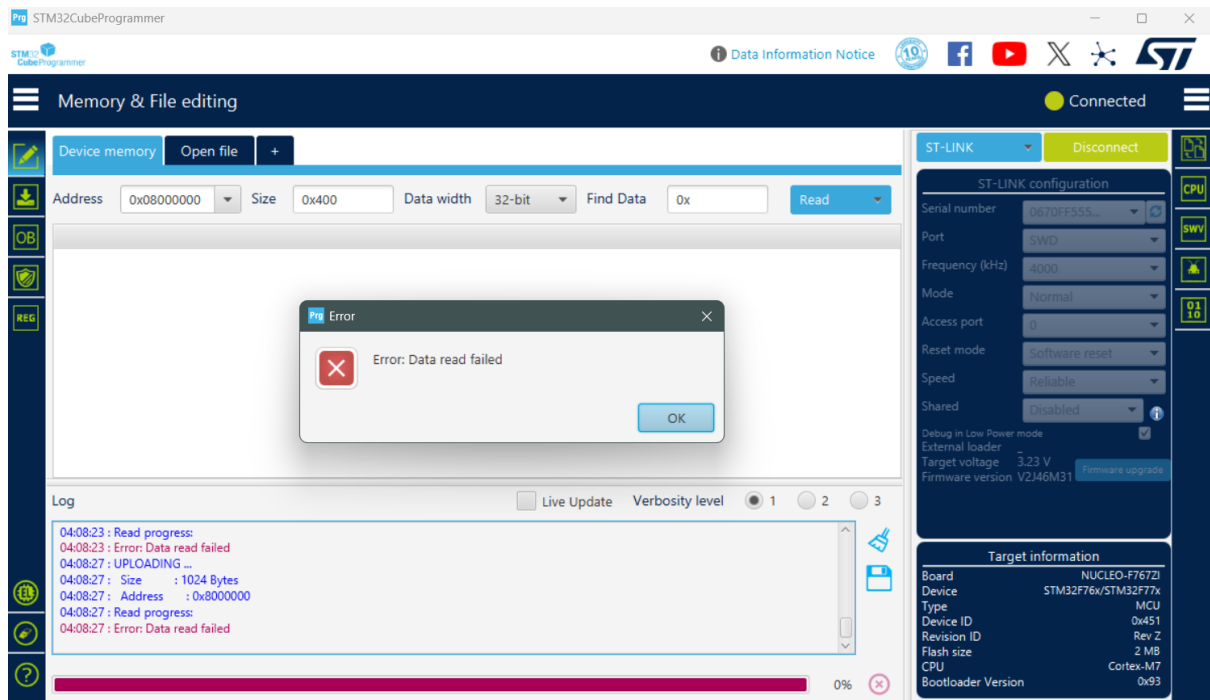


Figure 9- Read out protection enabled